

SEO PACKAGE

1. SEO Title

Maximum Product of Three Numbers in Python — The Negative Trap Every Beginner Falls For (LeetCode 628)

2. SEO Subtitle

Learn why the three biggest numbers don't always win, and how the "Two Corners" trick solves LeetCode 628 in a few clean lines of Python.

3. Featured Quote

"Being *large* and giving the *largest product* are two different things the moment a negative number joins the list."

4. Meta Description

Solve LeetCode 628 Maximum Product of Three Numbers in Python. Master the negative-number trap with the Two Corners trick, clean code, and a dry run.

5. URL Slug

maximum-product-of-three-numbers-python-leetcode

6. Keywords

Maximum Product of Three Numbers, LeetCode 628, Python coding interview, beginner LeetCode problems, negative number multiplication, sorting algorithm Python, two corners trick, max product array, coding interview Python, data structures and algorithms, DSA for beginners, Python sort solution, array problems LeetCode, product of three integers, greedy array problems, time complexity analysis, Python max function, interview preparation, algorithm intuition, edge cases coding

7. Tags

python, leetcode, coding-interview, algorithms, data-structures, arrays, sorting, beginner, dsa, software-engineering, problem-solving, interview-prep, leetcode-628

MAXIMUM PRODUCT OF THREE NUMBERS

8. Introduction

There is a special category of coding interview questions that look trivial and then quietly punish anyone who trusts their first instinct. **Maximum Product of Three Numbers** (LeetCode 628) is a perfect example. The task sounds like something you could solve in your head: pick three numbers from a list so their product is as large as possible.

Most people, including strong engineers, answer instantly: "*Just take the three biggest numbers.*" And for many inputs, that answer is correct. Then the interviewer slides one small list across the table, your confident solution returns the wrong number, and suddenly you are debugging under pressure.

That is exactly why this problem shows up so often in beginner and warm-up interviews. It is not testing whether you can multiply. It is testing whether you can **reason about negative numbers**, whether you notice a hidden case before it bites you, and whether you can turn that insight into clean, correct **Python**.

If you are learning **algorithms and data structures**, this problem is a gift. It teaches a mental habit that carries into far harder questions: the best answer often hides at the *extremes* of your data, not just the top. Let's build that intuition from the ground up.

9. Problem Overview

You are given a list of whole numbers (integers). These numbers can be positive, negative, or zero. Your job is to choose **exactly three** of them so that when you multiply those three together, the result is the **largest value possible**. You then return that product.

A few things worth stating plainly:

- The word **product** simply means the result of multiplying numbers. The product of 2, 3, and 4 is $2 \times 3 \times 4 = 24$.
- You must pick **three** numbers — no more, no fewer.
- The numbers may repeat in value, and the list can contain negatives and zeros.
- You are guaranteed the list has at least three numbers, so a valid answer always exists.

The whole difficulty lives in one word: *largest*. Our brains read "largest" and picture the biggest-looking numbers. But once negatives are allowed, the largest **product** can come from numbers that look small.

10. Example

Let's start with the friendly case and then meet the trap.

Example 1 — all positive:

Input: [1, 2, 3, 4]
Output: 24

The three largest numbers are 2, 3, 4, and $2 \times 3 \times 4 = 24$. Nothing surprising here — the biggest numbers win.

Example 2 — the negative trap:

Input: [-9, -8, 1, 2, 3]
Output: 216

Instinct says take 1, 2, 3, which gives 6. But look at the two negatives. $-9 \times -8 = 72$, because the two minus signs cancel and produce a **positive**. Then $72 \times 3 = 216$. That crushes 6.

Example 3 — all negative:

Input: [-1, -2, -3]
Output: -6

Here every product is negative, so we want the product closest to zero. The three numbers closest to zero are -1, -2, -3, and $-1 \times -2 \times -3 = -6$. That is the best we can do, and a correct solution must still return it.

These three examples map out the entire problem. Everything else is a variation of them.

11. Intuition

Here is how an experienced engineer arrives at the answer — not by memorizing a formula, but by asking "where could the winning combination possibly come from?"

Start with a simpler question. If all numbers were positive, where would the best product be? Clearly among the **three largest** numbers. Big times big times big equals biggest. Easy.

Now introduce negatives. A negative times a negative is a positive, and if those two negatives are large in magnitude (like `-9` and `-8`), their product is a *large* positive. Multiply that by the single biggest number in the list, and you may beat the three-largest combination entirely.

So there are really only two shapes a winning answer can take:

1. **The three largest numbers.** This wins when the big numbers dominate (all positive, or negatives too small to matter).
2. **The two smallest numbers × the largest number.** The two smallest are the most negative ones; multiplied together they flip positive, then boosted by the biggest number.

Notice what both candidates have in common: they only ever use numbers from the **two ends** of a sorted list. Nothing in the middle can ever help — the middle is neither big enough to lift the product nor negative enough to flip it. I like to call this the **Two Corners** insight: sort the list, then the answer is hiding in the left corner, the right corner, or a mix of both.

Once you see it that way, the code writes itself.

12. Brute Force Solution

Idea: If we're unsure which trio wins, just try *every* trio. Check all possible groups of three numbers, compute each product, and keep the maximum.

Algorithm: 1. Loop over every choice of three distinct positions `i < j < k`. 2. Multiply `nums[i] * nums[j] * nums[k]`. 3. Track the largest product seen.

```
from itertools import combinations

def maximum_product_brute(nums):
    best = float("-inf")
    for a, b, c in combinations(nums, 3):
        best = max(best, a * b * c)
    return best
```

Advantages: - Impossible to get wrong logically — it literally tests everything. - Great sanity-check reference to compare a faster solution against.

Disadvantages: - Painfully slow as the list grows. The number of triples explodes. - Does unnecessary work; most trios are obviously hopeless.

Complexity: - **Time:** $O(n^3)$ — three nested choices over n elements. - **Space:** $O(1)$ extra (ignoring the combinations generator).

For a list of a few thousand numbers, n^3 becomes billions of operations. We can do far better by using the structure we discovered.

13. Optimal Solution

The Two Corners insight gives us a shortcut. Sort the list, and every number we could possibly need lines up neatly at the two ends.

After sorting, the list looks like this:

```
[ smallest ... .. largest ]
  nums[0]  nums[1]          nums[-2] nums[-1]
  ↑ left corner          right corner ↑
```

Now we form just **two candidates**:

Candidate	Numbers used	Wins when...
A — three largest	$\text{nums}[-1] \times \text{nums}[-2] \times \text{nums}[-3]$	Big positives dominate, or all-negative lists
B — two smallest $\times$ largest	$\text{nums}[0] \times \text{nums}[1] \times \text{nums}[-1]$	Two large negatives flip to a big positive

The final answer is simply the larger of the two:

```
answer = max(A, B)
```

That's the entire algorithm. Sort, compute two products, return the bigger one. No nested loops, no special cases for negatives or zeros — the two corners handle all of it automatically.

Why is checking only two candidates enough? Because any winning product must maximize magnitude *and* land on a positive sign. Magnitude comes from the extremes (largest positives or most-negative values), and the sign is fixed by how many negatives you include. Candidate A covers "use the top three," Candidate B covers "recruit the negative pair." No third shape can ever beat both.

14. Python Code

```
def maximum_product(nums: list[int]) -> int:
    """Return the largest product obtainable from any three numbers in nums."""
    nums.sort()

    # Candidate A: the three largest numbers.
    top_three = nums[-1] * nums[-2] * nums[-3]

    # Candidate B: the two smallest (possibly big negatives) times the largest.
    two_smallest_and_largest = nums[0] * nums[1] * nums[-1]

    return max(top_three, two_smallest_and_largest)
```

Clean, readable, and correct across positives, negatives, and zeros. This is the version I'd reach for in an interview.

15. Code Walkthrough

- `nums.sort()` — Orders the list from smallest to largest. This is the key move: it pushes the biggest numbers to the right end and the smallest (including big negatives) to the left end, so both corners are easy to reach.
- `nums[-1] * nums[-2] * nums[-3]` — In Python, `-1` is the last item, `-2` the second-to-last, and `-3` the third-to-last. This is Candidate A, the product of the three largest numbers.
- `nums[0] * nums[1] * nums[-1]` — `0` is the first item and `1` is the second, so these are the two smallest numbers. Multiplied by `nums[-1]`, the largest, this is Candidate B — the negative-pair play.
- `return max(...)` — `max` is Python's built-in helper that returns the larger of two values. Whichever candidate is bigger is our answer.

That is the whole toolbox: `sort` and `max`. Two built-ins do all the heavy lifting.

16. Dry Run

Let's trace the tricky input `[-9, -8, 1, 2, 3]` step by step.

Step 1 — sort:

```
[-9, -8, 1, 2, 3] (already sorted here)
```

Step 2 — Candidate A (three largest):

```
nums[-1] * nums[-2] * nums[-3]
= 3 * 2 * 1
= 6
```

Step 3 — Candidate B (two smallest × largest):

```
nums[0] * nums[1] * nums[-1]
= -9 * -8 * 3
= 72 * 3
= 216
```

Step 4 — take the max:

```
max(6, 216) = 216
```

The function returns **216**, correctly catching the negative-pair combination that the naive "three biggest" answer would have missed.

Try the same trace on `[-1, -2, -3]`: Candidate A is `-1 * -2 * -3 = -6`, Candidate B is `-3 * -2 * -1 = -6`, and `max(-6, -6) = -6`. Both corners agree, and the answer is correct even when everything is negative.

17. Complexity Analysis

- **Time:** $O(n \log n)$ — The work is dominated by sorting the list, which takes $O(n \log n)$. Computing the two candidate products and taking their max is constant work, $O(1)$. So sorting sets the pace.
- **Space:** $O(1)$ to $O(n)$ — Python's `list.sort()` sorts in place, so no extra list is created. Depending on how you count the internal working memory of the sort (Timsort), you may see it described as $O(n)$ in the worst case, but no additional data structures grow with the input in our own code.

Compared to the brute force's $O(n^3)$, this is a dramatic improvement — from billions of operations down to a quick sort and a handful of multiplications.

18. Alternative Solutions

There's a way to avoid sorting entirely. Instead of ordering the whole list, **scan it once** and track only the values you actually need:

- the **three largest** numbers seen so far ($\text{max1} \geq \text{max2} \geq \text{max3}$), and
- the **two smallest** numbers seen so far ($\text{min1} \leq \text{min2}$).

```
def maximum_product_no_sort(nums: list[int]) -> int:
    max1 = max2 = max3 = float("-inf")  # three largest
    min1 = min2 = float("inf")          # two smallest

    for n in nums:
        # update the three largest
        if n >= max1:
            max1, max2, max3 = n, max1, max2
        elif n >= max2:
            max2, max3 = n, max2
        elif n >= max3:
            max3 = n
        # update the two smallest
        if n <= min1:
            min1, min2 = n, min1
        elif n <= min2:
            min2 = n

    return max(max1 * max2 * max3, min1 * min2 * max1)
```

Comparison:

	Sort-based	One-pass scan
Time	$O(n \log n)$	$O(n)$
Lines of code ~4		~15
Risk of bugs	Very low	Higher (fiddly updates)
Best for	Beginners, interviews	Very large inputs

My honest advice for a beginner: use sorting. The scan is technically faster, but on realistic input the difference is negligible, and the sort version is nearly impossible to get wrong. Simple and correct beats clever and fragile every time. Reach for the scan only when profiling proves the sort is a real bottleneck.

19. Edge Cases

- **All negative numbers** — e.g. $[-1, -2, -3]$. The answer is negative (-6), and Candidate A (three largest = three closest to zero) handles it.

- **Two negatives, rest positive** — e.g. `[-10, -10, 1, 3, 2]`. Candidate B (`-10 * -10 * 3 = 300`) wins. This is the classic trap.
- **Contains zeros** — e.g. `[-4, -3, 0, 2]`. Zeros are handled automatically; the corners simply won't pick a zero unless it genuinely gives the best product.
- **Minimum size** — exactly three numbers. Both candidates use those same three values, so the answer is just their product.
- **Duplicates** — e.g. `[3, 3, 3]`. No special handling needed; duplicates are ordinary values.
- **Large magnitudes** — Python integers are arbitrary precision, so there's no overflow risk (a genuine concern in languages like Java or C++).

20. Common Mistakes

1. **Only checking the three largest numbers.** This ignores the negative pair and fails on inputs like `[-9, -8, 1, 2, 3]`. The single most common bug in this problem.
2. **Forgetting that two negatives make a positive.** If you don't internalize this, the whole trick is invisible and your solution looks "obviously correct" while being wrong.
3. **Comparing the wrong corners.** Some people multiply the two smallest by the *smallest* largest instead of the actual largest (`nums[-1]`). Candidate B must use the single biggest number.
4. **Assuming the answer is always positive.** On all-negative lists the maximum product is negative. Initializing your "best" to `0` instead of `-inf` quietly breaks these cases.
5. **Over-optimizing too early.** Jumping straight to the no-sort scan and mismanaging the min/max updates. Fragile edge-case handling introduces bugs the sort version never has.
6. **Reaching outside the list bounds.** Using `nums[-3]` without confirming at least three elements exist — fine here because the problem guarantees it, but a habit worth being deliberate about.

21. Interview Questions

Expect follow-ups like these:

- *"Can you solve it without sorting?"* → Present the one-pass min/max scan and explain the `O(n)` vs `O(n log n)` trade-off.
- *"What if you had to find the maximum product of k numbers instead of three?"* → Now it's a genuine trade-off between the largest and smallest elements at both ends; sorting plus checking combinations of the ends generalizes the idea.

- *"How does your solution behave with all negative numbers?"* → Explain that Candidate A (three closest to zero) handles it.
- *"Would this work in a language with fixed-size integers?"* → Discuss integer overflow — a real concern in Java/C++, but not in Python.
- *"Can you prove only two candidates are ever needed?"* → Argue from magnitude and sign: the extremes maximize magnitude, and the number of negatives fixes the sign.

22. Similar Problems

- [Maximum Subarray \(LeetCode 53\)](#) — Also about maximizing a combination while carefully handling negatives; introduces Kadane's algorithm.
- [Maximum Product Subarray \(LeetCode 152\)](#) — The direct sequel: the same "two negatives flip to a positive" idea, but over contiguous subarrays. The corner trick evolves into tracking both max and min running products.
- [Best Time to Buy and Sell Stock \(LeetCode 121\)](#) — Another "answer lives at the extremes" problem, tracking a running minimum.
- [Third Maximum Number \(LeetCode 414\)](#) — Practices the same "track the top few values in one pass" muscle used in our no-sort variant.

They're related because each rewards the same instinct: the optimal answer tends to hide at the boundaries of your data, and negatives can invert your assumptions.

23. Key Takeaways

- **"Largest number" and "largest product" are not the same** once negatives enter the picture.
- **Two negatives multiply into a positive** — the single insight the whole problem is built around.
- After sorting, the winning trio always hides in the **two corners**: the three largest, or the two smallest times the largest.
- **Sort + max** gives a clean $O(n \log n)$ solution in four lines. A one-pass $O(n)$ scan exists but is easy to fumble.
- Prefer **simple and correct** over clever and fragile, especially in interviews.

24. Watch the Video

Reading builds understanding, but watching the corners light up on a sorted list makes it *click*. In the video, we build this solution line by line, watch the naive approach break on one sneaky list, and walk through the exact test cases that fool people. If this article helped, the video will lock it in.

▶ **Watch it here:** {LINK}

25. About the Series

This is part of **Fun with Learning Technology** — a beginner-friendly series that turns intimidating coding problems into calm, step-by-step lessons. Each entry starts gentle, builds intuition *before* code, and never leaves you memorizing a solution you don't understand. A new problem drops daily, and this same **Two Corners** trick cracks an even tougher one tomorrow. If you're learning **Python**, **data structures**, or preparing for **coding interviews**, follow along — one small, confident step at a time.

26. Call To Action

If this made the negative trap finally make sense:

- 👍 **Like** the video on YouTube — it genuinely helps the channel grow.
- 🔔 **Subscribe** so you catch tomorrow's problem, where the corner trick returns.
- 📄 **Subscribe on Substack** for the written deep-dives like this one.
- 💬 **Comment** with your answer to the practice case `[-5, -4, 1, 2, 3]` — did you get it?
- ➦ **Share** this with a friend who's grinding LeetCode.

Reaching the end is the hardest part, and you did it. Tapping like is the easy part. See you tomorrow.

SOCIAL MEDIA

LinkedIn Post

Here's a LeetCode problem that quietly humbles strong engineers: **Maximum Product of Three Numbers**.

The task looks trivial — pick three numbers from a list whose product is the largest possible. Almost everyone answers instantly: "Just grab the three biggest numbers."

And for positive-only lists, that's perfectly correct. Then you hit a list like `[-9, -8, 1, 2, 3]` :

- Three biggest $\rightarrow 1 \times 2 \times 3 = 6$ • Two negatives $\rightarrow (-9) \times (-8) \times 3 = 216$

The two smallest numbers crush the three biggest, because two negatives multiply into a positive. That single fact is the entire problem.

Once you see it, the solution is elegant. Sort the list, and the answer always hides at the two ends:

1. The three largest numbers, OR
2. The two smallest $\times$ the largest

Return whichever is bigger. Four lines of Python, $O(n \log n)$, handles negatives and zeros automatically:

```
def maximum_product(nums):
    nums.sort()
    return max(nums[-1]*nums[-2]*nums[-3],
               nums[0]*nums[1]*nums[-1])
```

The lesson goes beyond this one problem: **the optimal answer often lives at the extremes of your data, not just the top.** That instinct carries into Maximum Product Subarray, Best Time to Buy and Sell Stock, and plenty of harder questions.

There's also a faster $O(n)$ one-pass version — but for a beginner, the sort is nearly impossible to get wrong. Simple and correct beats clever and fragile.

What's a problem that looked easy until one test case humbled you?

Python #CodingInterview #Algorithms #DataStructures #SoftwareEngineering

Twitter/X Thread

1/ LeetCode "Maximum Product of Three Numbers" looks trivial: pick 3 numbers with the biggest product.

Almost everyone gets it wrong on the first try. Here's the trap 🪤

2/ The instinct: "Just take the three biggest numbers."

For `[1, 2, 3, 4]` $\rightarrow 2 \times 3 \times 4 = 24$. Correct!

Feels solved. It isn't.

3/ Now try `[-9, -8, 1, 2, 3]`.

Three biggest $\rightarrow 1 \times 2 \times 3 = 6$.

But watch the negatives...

4/ $(-9) \times (-8) = 72$. Two minus signs cancel into a positive.

$72 \times 3 = 216$.

216 crushes 6. The two SMALLEST numbers won.

5/ This is the whole problem: two negatives multiply into a big positive.

"Largest number" $\neq$ "largest product" once negatives exist.

6/ The fix — I call it Two Corners.

Sort the list. The answer always hides at the two ends:

A) three largest B) two smallest $\times$ the largest

7/

```
def maximum_product(nums):  
    nums.sort()  
    return max(nums[-1]*nums[-2]*nums[-3],  
               nums[0]*nums[1]*nums[-1])
```

4 lines. $O(n \log n)$. Done.

8/ Why only 2 candidates? Magnitude comes from the extremes; sign is fixed by how many negatives you include.

Nothing in the middle can ever win.

9/ Bonus: all-negative lists like `[-1, -2, -3]` ?

Answer is -6 (the three closest to zero). The same code handles it — no special case.

10/ There's a faster $O(n)$ one-pass version too, but for beginners the sort is nearly bug-proof.

Simple + correct $>$ clever + fragile.

Save this for your next interview. 🚀

Facebook Post

Ever seen a coding problem that looks *way* too easy... and then bites you? 🤪

"Pick three numbers from a list with the biggest product." Simple, right? Just grab the three biggest.

Except — try the list `[-9, -8, 1, 2, 3]`. The three biggest give you 6. But $(-9) \times (-8) \times 3 = 216$! Two negatives multiply into a positive, and suddenly the *smallest* numbers win.

The clean trick: sort the list, then just compare the three largest vs. the two smallest times the largest. Four lines of Python and you're done. 🔄

Full step-by-step walkthrough (built line by line, super beginner-friendly) is up now. Give it a watch and tell me if you'd have caught the trap! 👉

Reddit Summary

The negative-number trap in "Maximum Product of Three Numbers" (LeetCode 628)

Sharing a clean way to think about this one, since it trips up a lot of beginners.

The naive approach — "sort and take the three largest" — works for all-positive lists but fails when big negatives are present. Example: `[-9, -8, 1, 2, 3]`. Three largest give 6, but $(-9) * (-8) * 3 = 216$, because two negatives produce a positive.

Key insight: after sorting, the optimal trio always comes from the two ends of the list. There are only two candidates:

1. `nums[-1] * nums[-2] * nums[-3]` (three largest)
2. `nums[0] * nums[1] * nums[-1]` (two smallest × largest)

Return the max of the two. It's $O(n \log n)$, and it handles all-negative lists and zeros with no special casing.

```
def maximum_product(nums):
    nums.sort()
    return max(nums[-1]*nums[-2]*nums[-3],
               nums[0]*nums[1]*nums[-1])
```

There's an $O(n)$ one-pass version that tracks the top-3 and bottom-2 as it scans, if you want to avoid sorting. Slightly faster, noticeably easier to get wrong on the update logic — worth knowing but not worth the bug risk unless the input is huge.

The transferable lesson: optimal answers often sit at the extremes of the data, and negatives can invert your assumptions. Same idea shows up in Maximum Product Subarray (152).

GITHUB README

Maximum Product of Three Numbers (LeetCode 628)

Find three numbers in a list whose product is the largest possible, and return that product. A begi

Problem

Given an integer array `nums`, choose exactly three numbers whose product is maximized, and return that maximum product. The array may contain positive numbers, negative numbers, and zeros, and is guaranteed to have at least three elements.

Examples

Input	Output	Why
[1, 2, 3, 4]	24	$2 \times 3 \times 4$
[-9, -8, 1, 2, 3]	216	$(-9) \times (-8) \times 3$ – two negatives flip positive
[-1, -2, -3]	-6	Three values closest to zero

Intuition

The tempting answer is "take the three biggest numbers." That works for all-positive lists, but breaks the moment large negatives appear, because **two negatives multiply into a positive**.

After sorting, every candidate for the answer lives at the two ends of the list ("Two Corners"):

1. The **three largest** numbers, or
2. The **two smallest** numbers (possibly big negatives) times the **largest**.

Nothing in the middle can ever win – the extremes control both magnitude and sign. So we only ever compare two products.

Approach

1. Sort the array.
2. Compute Candidate A: `nums[-1] * nums[-2] * nums[-3]`.
3. Compute Candidate B: `nums[0] * nums[1] * nums[-1]`.
4. Return `max(A, B)`.

This handles negatives, zeros, and all-negative lists with no special cases.

Python Solution

```

python
def maximum_product(nums: list[int]) -> int:
    """Return the largest product obtainable from any three numbers in nums."""
    nums.sort()
    top_three = nums[-1] * nums[-2] * nums[-3]          # three largest
    two_smallest_and_largest = nums[0] * nums[1] * nums[-1] # negative pair play
    return max(top_three, two_smallest_and_largest)

```

Optional: O(n) one-pass (no sort)

```

def maximum_product_no_sort(nums: list[int]) -> int:
    max1 = max2 = max3 = float("-inf")
    min1 = min2 = float("inf")
    for n in nums:
        if n >= max1: max1, max2, max3 = n, max1, max2
        elif n >= max2: max2, max3 = n, max2
        elif n >= max3: max3 = n
        if n <= min1: min1, min2 = n, min1
        elif n <= min2: min2 = n
    return max(max1 * max2 * max3, min1 * min2 * max1)

```

Complexity

Solution	Time	Space
Sort-based	$O(n \log n)$	$O(1)$ extra
One-pass scan	$O(n)$	$O(1)$
Brute force	$O(n^3)$	$O(1)$

Video

 Watch the full walkthrough: {LINK}

Article

Full write-up with dry runs, edge cases, and common mistakes is available in the accompanying article. Part of the **Fun with Learning Technology** series. ``

Quick Review

When you need the max product of 3 numbers from an array, what's the trick?

Sort, then compare two candidates: the top three values, OR the two smallest (most negative) times the largest. Two negatives make a positive.

Why can't you just multiply the three largest numbers?

Two large negative numbers multiply to a big positive. Pairing them with the max can beat the product of the top three positives.

Time complexity of the sort-based solution?

$O(n \log n)$ from sorting. A one-pass approach tracking the 3 largest and 2 smallest values is $O(n)$.

Space complexity?

$O(1)$ extra if sorting in place (or $O(n)$ depending on sort). The linear scan tracking min/max values uses $O(1)$.

What's the classic edge case that breaks naïve solutions?

Arrays with negative numbers, e.g. $[-10, -10, 1, 3, 2]$: answer is $(-10)(-10)(3)=300$, not $3 \cdot 2 \cdot 1=6$.

Follow-up: how to solve without sorting?

Single pass tracking the three largest and two smallest values, then compare $\text{max1} \cdot \text{max2} \cdot \text{max3}$ vs $\text{min1} \cdot \text{min2} \cdot \text{max1}$. $O(n)$ time.